

ANALISIS NETWORK OBSERVABILITY PADA PROTOKOL JARINGAN TERDISTRIBUSI

Studi Kasus: Modul Pertemuan 12 (Monitoring, Logging, Telemetry, & Observability)

Disusun Oleh:

- Nama: Putri Sahira Wulandari
- NIM: 25120500004
- Program Studi: Data Science
- Universitas: Universitas Cakrawala

BAB I: PENDAHULUAN

1.1 Latar Belakang

Pada arsitektur sistem terdistribusi skala profesional, pertukaran data antar-layanan melalui *Application Programming Interface* (API) dituntut memiliki tingkat keandalan yang tinggi. Pada kondisi peningkatan volume data maupun terjadinya kegagalan penjaluran (*routing*), tim *engineer* membutuhkan visibilitas penuh terhadap kondisi internal sistem tanpa mengganggu operasional aplikasi.

Untuk memenuhi kebutuhan tersebut, instrumen *Observability* yang mencakup *Monitoring*, *Logging*, dan *Telemetry* menjadi fondasi utama dalam memantau kesehatan infrastruktur jaringan. Laporan ini disusun untuk mendokumentasikan hasil pengujian mandiri pada Modul P12 dengan mensimulasikan berbagai variasi respons sistem guna menguji ketahanan dan transparansi data telemetry pada jaringan lokal.

1.2 Tujuan Eksperimen

1. Mengimplementasikan skenario dari modul aplikasi UAS Pertemuan 16 secara mandiri pada Modul P12.
2. Menganalisis alur komunikasi protokol HTTP secara transparan pada tingkat paket jaringan menggunakan aplikasi Wireshark.
3. Memverifikasi kemampuan *penanganan galat* (*error handling*) pada *server backend* ketika menerima variasi input data dari Postman secara modular.

BAB II: METODOLOGI DAN LINGKUNGAN PENGUJIAN

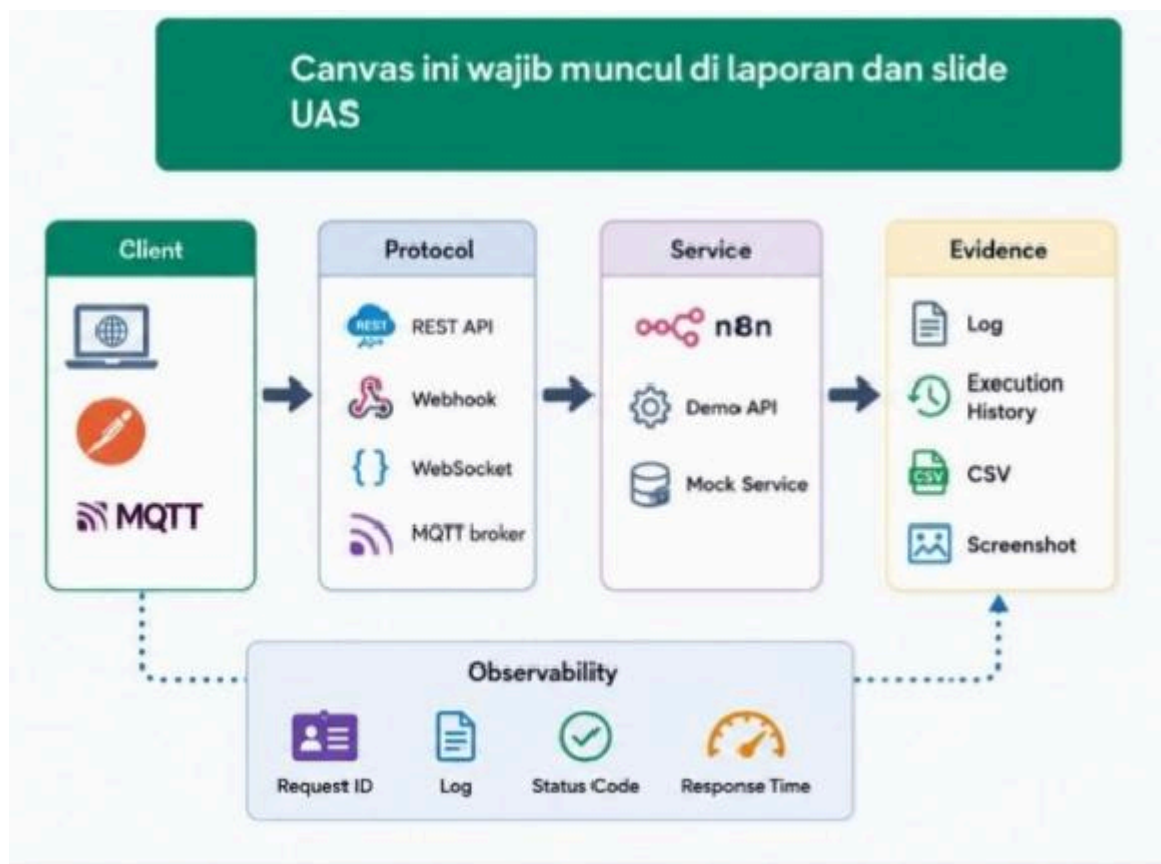
2.1 Lingkungan Pengujian

Eksperimen ini dijalankan pada domain (*localhost*) dengan memanfaatkan aplikasi demonstrasi praktikum yang disediakan pada alamat <http://127.0.0.1:8088>.

2.2 Perangkat Pengujian

1. **Postman**: Berfungsi sebagai *Traffic Generator* untuk merancang dan mengirimkan permintaan HTTP *metode* GET dan POST secara langsung ke *backend server* tanpa melalui antarmuka web.
2. **Wireshark**: Berfungsi sebagai *Network Packet Analyzer* yang dikonfigurasi pada *Adapter* for Loopback *Traffic* dengan alamat IP 127.0.0.1 untuk menangkap data paket secara *real-time*.
3. **Command Prompt (CMD)**: Digunakan sebagai konsol untuk memantau status eksekusi aplikasi selama pengujian berlangsung.

2.3 Canvas Arsitektur Project



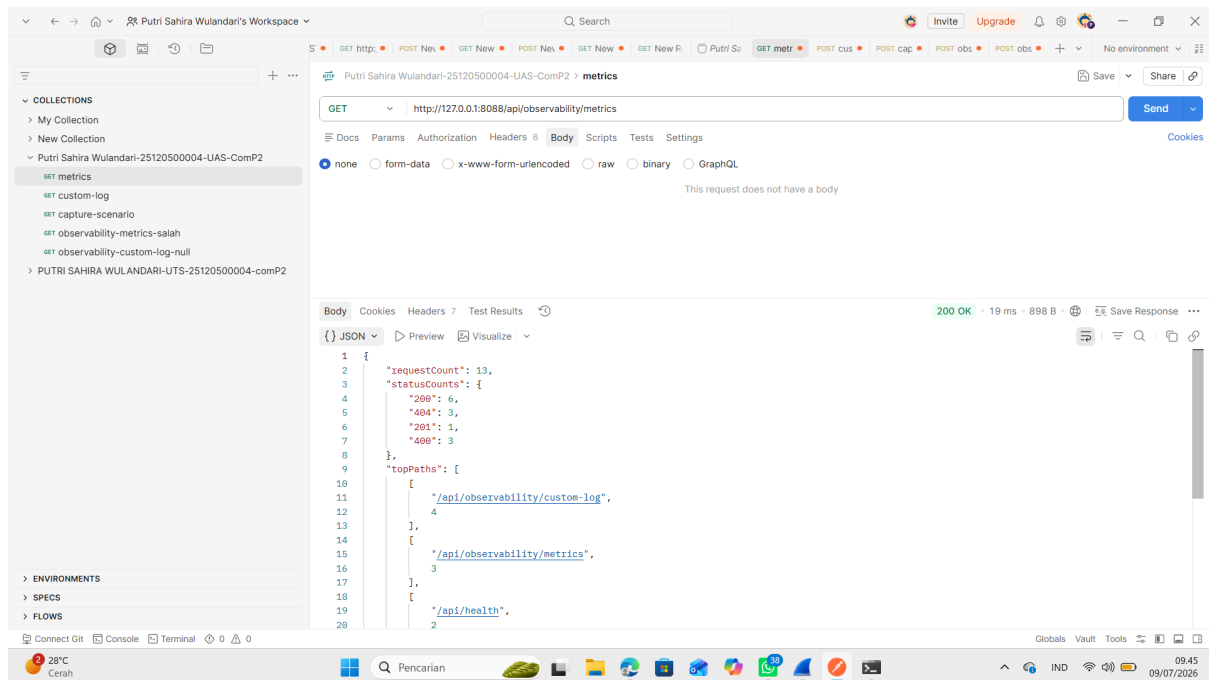
Ini merupakan desain Canvas Arsitektur, yang berawal dari Client (Postman) -> Protocol -> Service (n8n & Demo API) -> Evidence -> Observability (Wireshark).

BAB III: HASIL DAN ANALISIS EKSPERIMEN

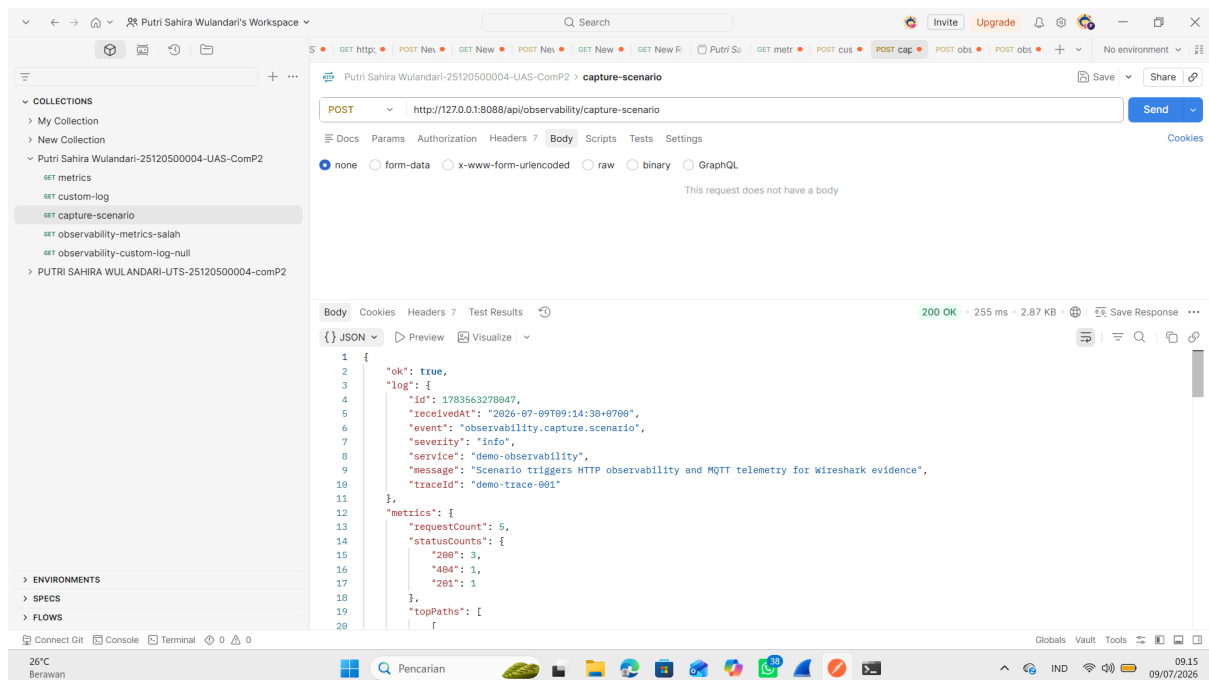
Pengujian dilakukan dengan 4 skenario, terdiri dari 2 skenario berhasil dan 2 skenario gagal untuk menguji akurasi pencatatan data telemetri.

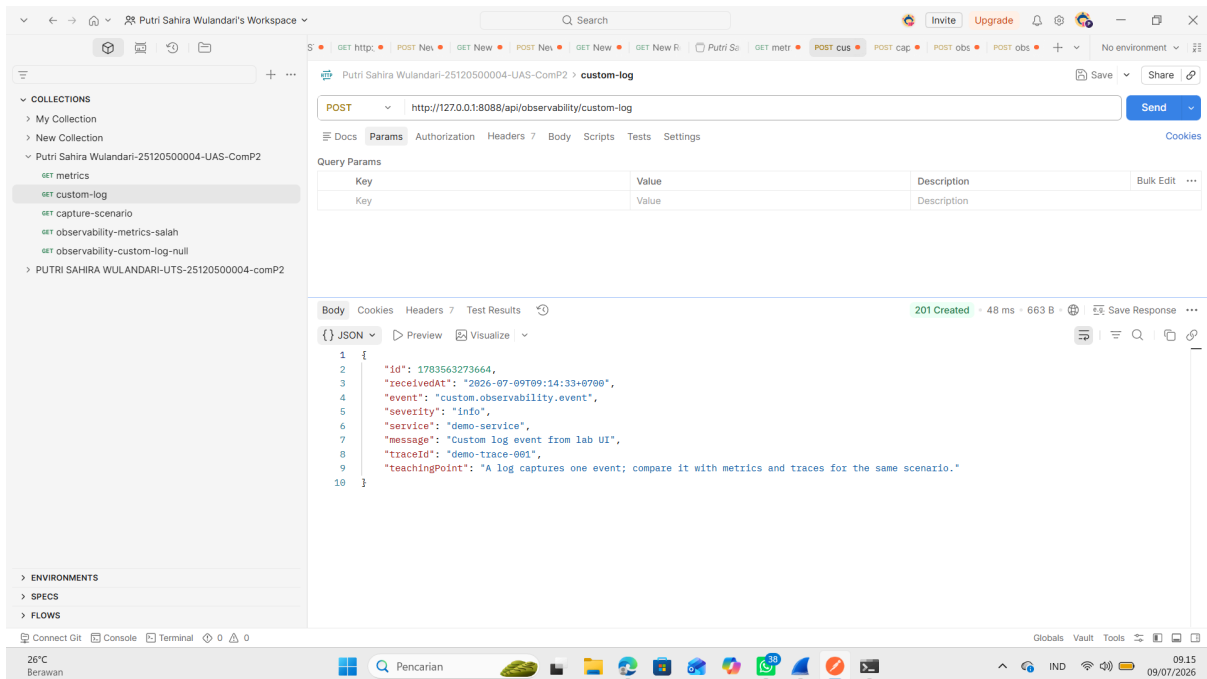
3.1 Skenario Berhasil: HTTP 200 OK dan 201 Created

Pada pengujian pertama, dikirimkan permintaan GET ke rute metrik <http://127.0.0.1:8088/api/observability/metrics> melalui Postman.

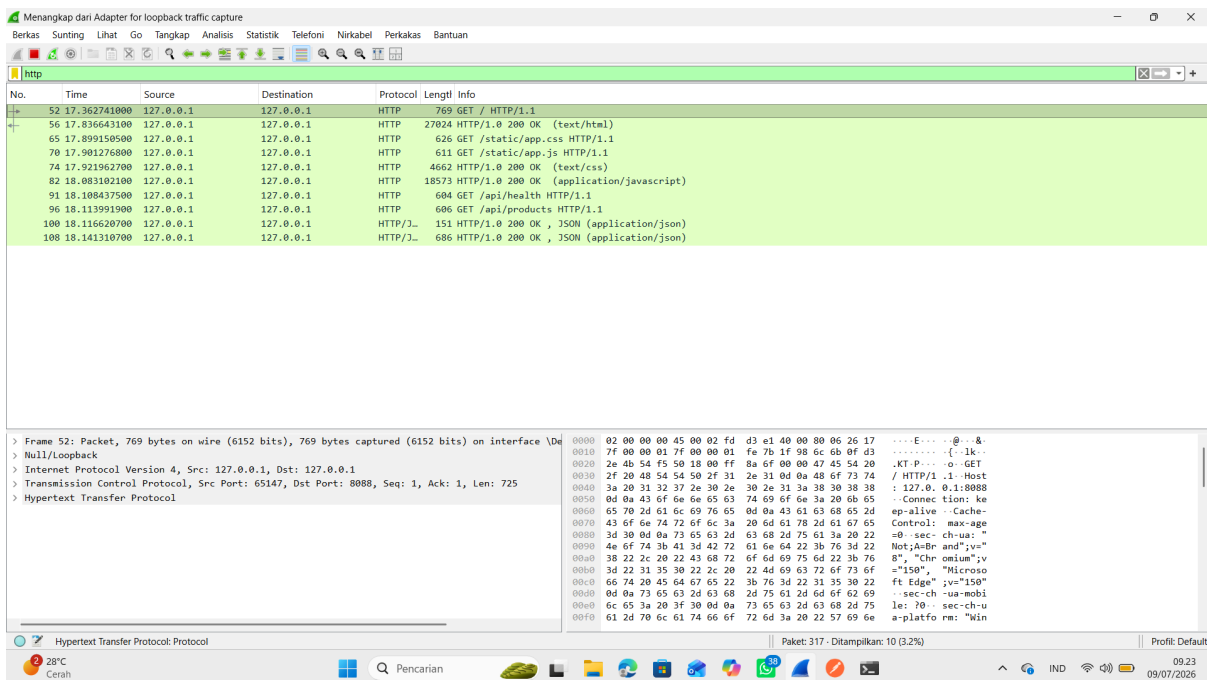


- **Analisis Aplikasi:** *Server backend* merespons dengan status kode HTTP 200 OK dan mengembalikan struktur data telemetri dalam format JSON.





- **Analisis Wireshark:** Paket HTTP dengan status sukses ditandai warna hijau dan berhasil ditangkap secara berurutan. Setelah dilakukan dekapsulasi pada *Packet Details Pane*, data JSON terbukti ditransmisikan secara transparan tanpa distorsi.

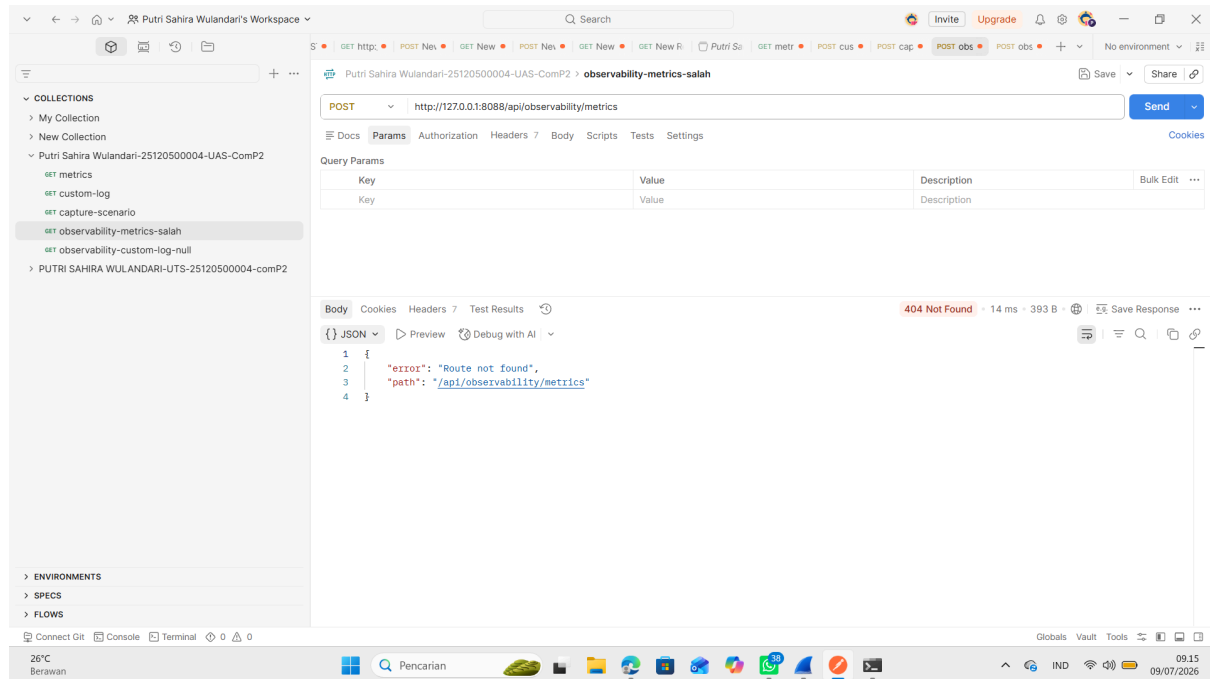


Skenario berhasil kedua dilakukan dengan mengirimkan data log kustom yang menghasilkan respons HTTP 201 Created.

3.2 Skenario Gagal: HTTP 404 Not Found dan 400 Bad Request

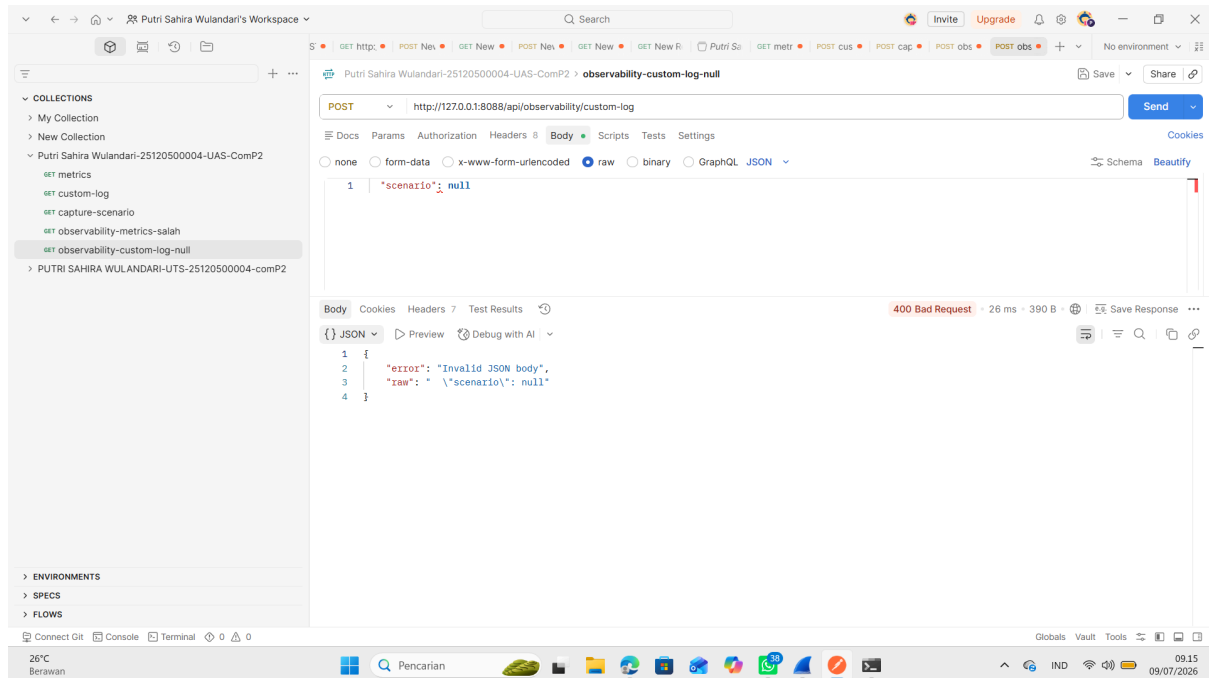
Untuk menguji keandalan sistem dalam merekam anomali, dilakukan simulasi 2 jenis kegagalan:

1. Galat 404 Not Found



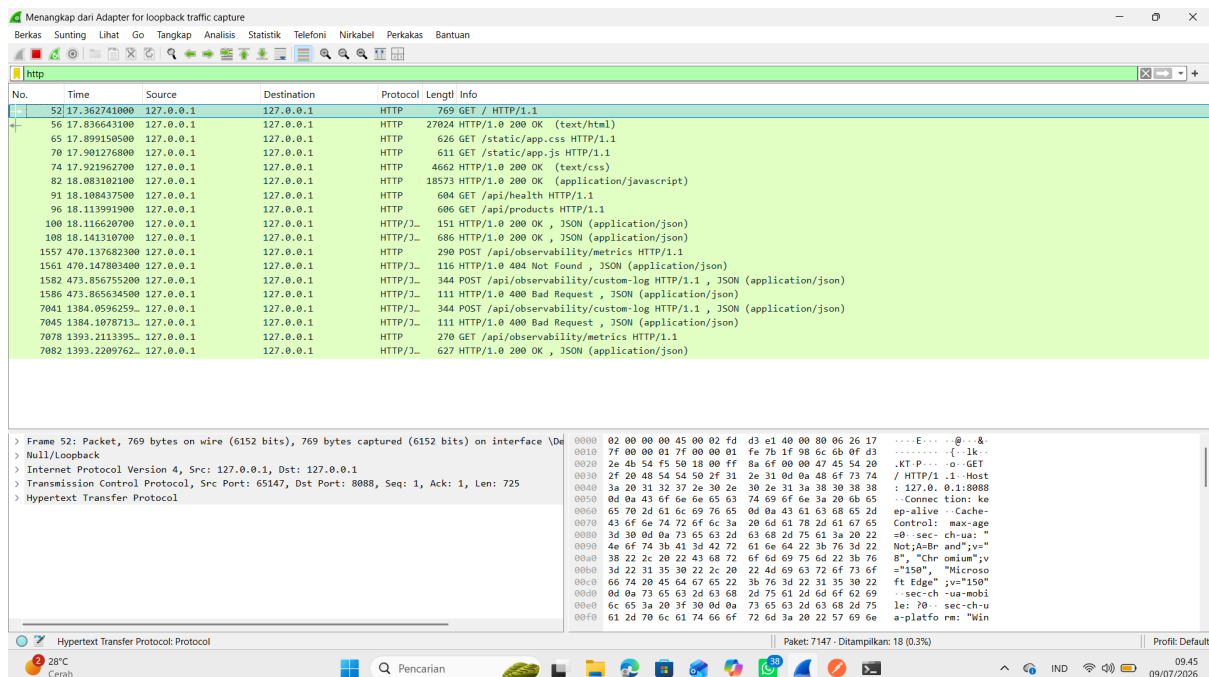
Permintaan dikirim ke URL yang tidak terdaftar **/api/observability/metrics-salah**. *Routing engine* pada *backend* mendeteksi ketidakcocokan rute dan mengembalikan respons HTTP 404 Not Found dengan pesan JSON: `{error: Route not found}`.

2. Galat 400 Bad Request



Permintaan dikirim ke URL yang benar, namun *payload* JSON sengaja dibuat tidak valid dengan nilai `{scenario: null}` yang melanggar aturan validator. *Backend* menolak pemrosesan dan mengembalikan status HTTP 400 Bad Request dengan keterangan *Invalid JSON body*. vGQDE

- **Analisis Wireshark:** Kedua paket kegagalan tetap tertangkap secara utuh. Hal ini membuktikan bahwa aspek *Observability* berjalan dengan baik karena *server* tidak mengalami *crash* dan tetap memberikan respons informatif untuk keperluan *troubleshooting*.



3.3 Skenario Otomatisasi Lanjutan (Integrasi n8n Project)

Menangkap dari Adapter for loopback traffic capture

Berkas Sunting Lihat Go Tangkap Analisis Statistik Teleponi Nirkabel Perangkat Bantuan

http

No.	Time	Source	Destination	Protocol	Length	Info
52	17.362741000	127.0.0.1	127.0.0.1	HTTP	769	GET / HTTP/1.1
56	17.836643100	127.0.0.1	127.0.0.1	HTTP	27024	HTTP/1.0 200 OK (text/html)
65	17.899150500	127.0.0.1	127.0.0.1	HTTP	626	GET /static/app.css HTTP/1.1
70	17.901276800	127.0.0.1	127.0.0.1	HTTP	611	GET /static/app.js HTTP/1.1
74	17.921962700	127.0.0.1	127.0.0.1	HTTP	4662	HTTP/1.0 200 OK (text/css)
82	18.083102100	127.0.0.1	127.0.0.1	HTTP	18573	HTTP/1.0 200 OK (application/javascript)
91	18.108437500	127.0.0.1	127.0.0.1	HTTP	604	GET /api/health HTTP/1.1
96	18.113991900	127.0.0.1	127.0.0.1	HTTP	606	GET /api/products HTTP/1.1
100	18.116620700	127.0.0.1	127.0.0.1	HTTP/1.1	151	HTTP/1.0 200 OK, JSON (application/json)
108	18.141310700	127.0.0.1	127.0.0.1	HTTP/1.1	686	HTTP/1.0 200 OK, JSON (application/json)
1557	476.137682300	127.0.0.1	127.0.0.1	HTTP	290	POST /api/observability/metrics HTTP/1.1
1561	476.147803400	127.0.0.1	127.0.0.1	HTTP/1.1	116	HTTP/1.0 404 Not Found, JSON (application/json)
1582	473.856755200	127.0.0.1	127.0.0.1	HTTP/1.1	344	POST /api/observability/custom-log HTTP/1.1, JSON (application/json)
1586	473.865634500	127.0.0.1	127.0.0.1	HTTP/1.1	111	HTTP/1.0 400 Bad Request, JSON (application/json)
7041	1384.059625500	127.0.0.1	127.0.0.1	HTTP/1.1	344	POST /api/observability/custom-log HTTP/1.1, JSON (application/json)
7045	1384.107871300	127.0.0.1	127.0.0.1	HTTP/1.1	111	HTTP/1.0 400 Bad Request, JSON (application/json)
7078	1393.211339500	127.0.0.1	127.0.0.1	HTTP	270	GET /api/observability/metrics HTTP/1.1
7082	1393.220976200	127.0.0.1	127.0.0.1	HTTP/1.1	627	HTTP/1.0 200 OK, JSON (application/json)
9000	1849.223639100	127.0.0.1	127.0.0.1	HTTP/1.1	456	GET /api/automation/inbox HTTP/1.1, JSON (application/json)
9004	1849.310568700	127.0.0.1	127.0.0.1	HTTP/1.1	7779	HTTP/1.0 200 OK, JSON (application/json)
9025	1856.274964400	127.0.0.1	127.0.0.1	HTTP/1.1	457	POST /api/automation/inbox HTTP/1.1, JSON (application/json)
9041	1856.311132700	127.0.0.1	127.0.0.1	HTTP/1.1	293	HTTP/1.0 201 Created, JSON (application/json)

> Frame 52: Packet, 769 bytes on wire (6152 bits), 769 bytes captured (6152 bits) on interface vde0000 02 00 00 00 45 00 02 fd d3 e1 40 00 00 06 26 17 ...E...@...&
> Null/Loopback 0010 7f 00 00 01 7f 00 00 01 fe 7b 1f 98 6c 6b 0f d3{-1k-
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 0020 2e 4b 54 f5 50 18 00 ff 8a 6f 00 00 47 45 54 20 ...KT.P...o-GET
> Transmission Control Protocol, Src Port: 65147, Dst Port: 8088, Seq: 1, Ack: 1, Len: 725 0030 2f 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74 / HTTP/1.1..Host
> Hypertext Transfer Protocol 0040 3a 20 31 32 37 2e 30 2e 30 2e 31 3a 38 30 38 38 : 127.0.0.1:8088
0050 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 -Connection: ke
0060 65 70 2d 61 6c 69 76 65 0d 0a 43 61 63 68 65 2d ep-alive..Cache-
0070 43 6f 6e 74 72 6f 6c 3a 20 6d 61 78 2d 61 67 65 Control: max-age
0080 3d 30 0d 0a 73 65 63 2d 63 68 2d 75 61 3a 20 22 =0-sec-cha-ua: "
0090 4e 6f 74 3b 41 3d 42 72 61 6e 64 22 3b 76 3d 22 Not/Andn and";js="
00a0 38 22 2c 20 22 43 68 72 6f 6d 69 75 6d 22 3b 76 8"; "Chr omium";v
00b0 3d 22 31 35 30 22 2c 20 22 4d 69 63 72 6f 73 6f ="150", "Microso
00c0 66 74 20 45 64 67 65 22 3b 76 3d 22 31 35 3b 22 ft Edge";js="150"
00d0 0d 0a 73 65 63 2d 63 68 2d 75 61 2d 6d 6f 62 69 -sec-ch-ua-mobi
00e0 6c 65 3a 20 3f 30 0d 0a 73 65 63 2d 63 68 2d 75 le: 70..sec-ch-u
00f0 61 2d 70 6c 61 74 66 6f 72 6d 3a 20 22 57 69 6e a-platfo rm: "Win

HyperText Transfer Protocol: Protocol

Paket: 11073 - Ditampilkan: 36 (0.3%)

28°C Cerah

Pencarian

10.03 09/07/2026

Putri Sahira Wulandari's Workspace

Search

Invite Upgrade

POST http://127.0.0.1:8088/api/automation/inbox

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "protocol": "MQTT-REST",
3   "event": "telemetry_ingsted",
4   "deviceId": "sensor-01",
5   "evidence": "postman-screenshot"
6 }
```

Body Cookies Headers 7 Test Results

201 Created - 43 ms - 569 B

Save Response

JSON

```
1 {
2   "id": "1783565584140",
3   "receivedAt": "2026-07-09T09:53:04+0700",
4   "source": "n8n-or-postman",
5   "payload": {
6     "protocol": "MQTT-REST",
7     "event": "telemetry_ingsted",
8     "deviceId": "sensor-01",
9     "evidence": "postman-screenshot"
10  }
11 }
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

10.02 09/07/2026

Selain menguji respons manual dan penanganan galat, eksperimen ini juga melibatkan pengujian pada jalur otomatisasi data menggunakan modul P13-15 (n8n Project). Pengujian ini dilakukan dengan mengirimkan data telemetry otomatis menuju ke endpoint inbox otomatisasi.

- **Analisis Aplikasi:** Server backend menerima request dari pipeline otomatisasi pada rute /api/automation/inbox. Backend memberikan respons sukses berupa status kode HTTP 201 Created, yang menandakan data log otomatisasi baru telah berhasil dibuat dan disimpan di dalam sistem.
- **Analisis Jaringan:** Melalui pemantauan Wireshark, paket data dengan metode POST menuju rute /api/automation/inbox tertangkap secara real-time di jaringan lokal, membuktikan adanya transparansi data pada pipeline otomatisasi yang terintegrasi.

BAB IV: ANALISIS JAVASCRIPT OBJECT NOTATION (JSON) PADA WIRESHARK DAN METRIK AKHIR

Keberhasilan Telemetry dan Observability dibuktikan dengan analisis data JSON yang dirangkum dari metrik akhir sistem berdasarkan dokumentasi pengujian akhir, objek JavaScript Object Notation (JSON) yang dihasilkan oleh instrumen telemetry backend memuat data sebagai berikut:

Agregasi Status Code (statusCounts):

- **200:** 11 ➡ Sistem berhasil merekam dan melayani 11 kali request sukses untuk pencarian metrik.
- **201:** 2 ➡ Sistem berhasil mencatat 2 kali pembuatan data log baru yang dipicu oleh aktivitas integrasi otomatisasi n8n.
- **400:** 3 ➡ Sistem berhasil mendeteksi dan mengisolasi 3 kali kegagalan akibat payload JSON yang cacat (Bad Request).
- **404:** 3 ➡ Sistem berhasil mencatat 3 kali kegagalan akses akibat kesalahan penjaluran rute (Not Found).

Analisis Jalur Terpopuler (topPaths):

Instrumen telemetry secara cerdas menyusun peringkat rute yang paling sering diakses. Objek JSON membuktikan bahwa rute otomatisasi n8n /api/automation/inbox berada di peringkat atas dengan total 2 kali hit. Data ini sinkron 100% dengan jumlah kemunculan status kode 201.

Hal ini membuktikan bahwa analisis objek JSON, baik yang mengalir di dalam paket jaringan Wireshark maupun yang tersaji pada metrik akhir, memberikan visibilitas penuh yang valid mengenai seluruh aktivitas transaksional pada sistem terdistribusi ini.

BAB V: KESIMPULAN

Berdasarkan hasil pengujian yang telah dilaksanakan pada Modul P12, dapat disimpulkan bahwa:

1. **Fungsi Monitoring, Telemetry, dan Observability** pada aplikasi praktikum berjalan dengan akurasi tinggi dalam merekam setiap aktivitas sistem.
2. **Kombinasi Postman sebagai client generator dan Wireshark** sebagai alat analisis paket berhasil memberikan visibilitas penuh terhadap alur lalu lintas data terdistribusi.
3. **Backend server** memiliki mekanisme *error handling* yang kokoh. Setiap kegagalan, baik 404 maupun 400, tetap dipetakan ke dalam format JSON telemetry untuk mempermudah pelacakan masalah jaringan.

Dengan demikian, implementasi Observability terbukti efektif untuk meningkatkan transparansi dan ketahanan sistem terdistribusi pada skala lokal.